

Contents

1	TechSapo Wall-Bounce システムフロー解説	1
1.1	目次	1
1.2	システム概要	1
1.3	全体フロー	2
1.4	ルーティング決定方式	6
1.5	Wall-Bounce の仕組み	8
1.6	LLM プロバイダー階層	13
1.7	具体的な処理例	16
1.8	品質保証の仕組み	21
1.9	まとめ	25
1.10	補足資料	25

1 TechSapo Wall-Bounce システムフロー解説

対象: 社内メンバー向け 作成日: 2025-10-07 バージョン: 1.0

1.1 目次

1. システム概要
 2. 全体フロー
 3. ルーティング決定方式
 4. Wall-Bounce の仕組み
 5. LLM プロバイダー階層
 6. 具体的な処理例
 7. 品質保証の仕組み
-

1.2 システム概要

TechSapo Wall-Bounce システムは、複数の LLM プロバイダーを協調動作させて高品質な回答を生成する AI 統合プラットフォームです。

1.2.1 コアコンセプト

「壁打ち (Wall-Bounce)」

テニスの壁打ちのように、複数の LLM に順次・並列で
問い合わせを行い、互いの回答を検証・改善することで
最終的に高品質な統合回答を生成する手法

1.2.2 主要な特徴

1. マルチ LLM 協調: 最低 2 つ以上の LLM を必ず使用
 2. 品質保証: コンセンサス 合意 スコアで品質を担保
 3. 自動ルーティング: タスクタイプに応じた最適な LLM 選択
 4. 階層化アーキテクチャ: Tier 0-4 の 5 段階で LLM を分類
-

1.3 全体フロー

1.3.1 エンドツーエンドフロー図

ユーザークエリ

「TypeScriptでバグ修正してください」

ステップ1: タスクタイプ判定

タスク分析エンジン

- ・ キーワード検出: "TypeScript", "バグ修正"
- ・ ドメイン判定: domain = "coding"
- ・ 複雑度評価: 通常タスク

判定結果: taskType = "coding", complexity = "normal"

ステップ2: LLMプロバイダー選択

ルーティングエンジン

```
IF taskType == "coding":  
    → Tier 2 Coding専用モデルを選択  
    → providers = [  
        "gpt-5-codex",      // OpenAI Codex  
        "qwen3-coder"       // Qwen3 Coder  
    ]  
  
ELSE IF taskType == "analysis":  
    → Tier 1/2 General モデルを選択  
    → providers = [  
        "gpt-5",            // GPT-5 標準  
        "gemini-2.0-flash"   // Gemini 2.0 Flash  
    ]
```

選択結果: ["gpt-5-codex", "qwen3-coder"]

ステップ3: Wall-Bounce 並列実行

クエリ + コンテキスト

GPT-5	Qwen3	Gemini
Codex	Coder	2.5 Pro
		(検証用)
Tier 2	Tier 2	Tier 1

Response 1	Response 2	Response 3
+ メタデータ	+ メタデータ	+ メタデータ

Quorum Judge
(コンセンサス判定)

- ・類似度計算
- ・品質スコア算出
- ・k=2 合意チェック

Consensus 0.6? (合意閾値)

YES	NO
-----	----

次ステップへ	追加LLM実行
--------	---------

ステップ4: アグリゲーター統合

アグリゲーター選択

IF complexity < 2:
→ aggregator = "sonnet-4.5" (Tier 3)

ELSE IF complexity ≥ 2:
→ aggregator = "opus-4.1" (Tier 4)

Claude Sonnet 4.5
(Default Aggregator)

Input:

- ・ 全プロバイダー回答
- ・ コンセンサススコア
- ・ 元のクエリ

Processing:

- ・ 矛盾の解消
- ・ 重複の統合
- ・ 最終推奨の生成

統合された最終回答
+ メタデータ
+ 品質スコア

最終レスポンス

```
{  
  "answer": "統合された回答テキスト",  
  "confidence": 0.85,  
  "consensus": 0.78,
```

```
"providersUsed": ["gpt-5-codex", "qwen3-coder", "gemini"],  
"aggregator": "sonnet-4.5"  
}
```

1.4 ルーティング決定方式

1.4.1 ルーティングの 3 段階判定

1.4.1.1 Phase 1: タスクタイプ判定

タスクタイプ判定ロジック

1. キーワード検出

Coding:

- "TypeScript", "バグ"
- "コード", "実装"
- "debug", "fix"

Analysis:

- "分析", "調査"
- "セキュリティ", "評価"
- "analyze", "review"

2. ドメイン明示

```
options.domain === "coding" → Coding  
options.domain === "analysis" → Analysis
```

3. デフォルト

不明な場合 → "basic" (汎用)

1.4.1.2 Phase 2: LLM プロバイダー選択

タスクタイプ別プロバイダーマッピング

Coding タスク:

Primary (Tier 2 - Coding特化):

gpt-5-codex

qwen3-coder

Validation (Tier 1 - 異なるベンダー):

gemini-2.0-flash

Analysis タスク:

Primary (Tier 1/2 - General):

gpt-5 (standard)

gemini-1.5-pro

Validation (Tier 1 - 高速):

gemini-2.0-flash

Basic タスク:

Cost-Efficient (Tier 1):

gemini-2.0-flash

gpt-5 (if needed)

Critical タスク:

All Available (Tier 1-4):

gemini-2.0-flash-thinking-exp

gpt-5 / gpt-5-codex

sonnet-4.5

opus-4.1

1.4.1.3 Phase 3: アグリゲーター選択

複雑度ベースのアグリゲーター選択

複雑度スコア計算:

```
score = 0
```

IF キーワード含む:

```
"architect", "design"
```

```
→ score += 1
```

IF 日本語キーワード含む:

```
"複雑な", "高度な", "詳細な"
```

```
→ score += 1
```

IF プロンプト長 > 500文字:

```
→ score += 1
```

IF 疑問符 3個:

```
→ score += 1
```

選択ロジック:

IF score < 2:

```
aggregator = "sonnet-4.5" ← Tier 3
```

(標準アグリゲーター)

ELSE IF score ≥ 2:

```
aggregator = "opus-4.1" ← Tier 4
```

(複雑タスク専用)

1.5 Wall-Bounce の仕組み

1.5.1 Wall-Bounce 実行フロー 詳細

ステップバイステップ詳細フロー

Step 1: プロバイダー並列実行

ユーザークエリ: "TypeScriptのバグを修正して"

↓

プロンプト生成 (Provider Guidance 付与)

GPT-5 Codex用:

「実装方針や設定変更の手順を具体的に示してください。必要に応じてコードスニペットやコマンド例を提示してください。」

+ ユーザークエリ

Qwen3 Coder用:

「TypeScriptのベストプラクティスに沿って具体的なコード例を示してください。差分形式や検証ステップがある場合は明記し、潜在的な副作用も指摘してください。」

+ ユーザークエリ

↓

Provider 1	Provider 2	Provider 3
------------	------------	------------

GPT-5	Qwen3	Gemini
Codex	Coder	2.5 Pro

[実行中..] [実行中..] [実行中..]

Response 1	Response 2	Response 3
"まず..."	"以下の..."	"TypeScript" "の修正..."
confidence: 0.85	confidence: 0.82	confidence: 0.78

Step 2: Quorum Judge (コンセンサス判定)

Quorum Judge Engine

設定:

- $k = 2$ (必要合意数)
- `scoreThreshold = 0.82`
- `abstainBelow = 0.5`

処理:

1. 類似度計算

Response1 vs Response2: 0.75

Response1 vs Response3: 0.68

Response2 vs Response3: 0.80

2. 合意グループ検出

Group A: [Response1, Response2]

(類似度 0.75, size=2)

3. Quorum達成チェック

Group A size (2) k (2)

→ Quorum達成

結果:

```
quorumAchieved: true
winningResponses: [Res1, Res2]
consensusScore: 0.75
metadata: {
  totalProviders: 3,
  agreedProviders: 2
}
```

↓

Consensus 0.6? → YES

↓

Step 3: アグリゲーター統合

Claude Sonnet 4.5 (Aggregator)

Input:

- 元のクエリ
- Winning Responses (2件)
- Consensus Score: 0.75
- Provider Metadata

Aggregation Prompt:

「以下の各LLM回答を統合し、
矛盾があれば整合させてください。
重複内容は統合し、最終的な
推奨行動・留意点・フォロー
アップを明確にしてください。」

Response 1 (GPT-5 Codex):

[内容...]

Response 2 (Qwen3 Coder):

[内容...]

Processing:

1. 矛盾点の検出・解消
2. 重複内容の統合
3. 最終推奨の生成
4. 品質スコアの付与

Output:

「TypeScriptのバグ修正について

【修正手順】

1. [統合された手順]
2. ...

【コード例】

[統合されたコード]

【注意点】

- [重要な留意事項]

【フォローアップ】

- [推奨される次のステップ]

↓

最終レスポンス生成

1.5.2 Early Stopping (アーリーストッピング)

Early Stopping フロー

設定:

- minProviders = 2
- maxProviders = 5
- k = 2 (必要合意数)

実行フロー:

Round 1: 2 providers実行

↓

Quorum達成? (size k)

YES → 終了 (高速完了)

NO → Round 2へ

Round 2: +1 provider実行 (total 3)

↓

Quorum達成?

YES → 終了

NO → Round 3へ

...

Round N: maxProvidersに達するまで繰り返し

メリット:

コスト削減 (不要なLLM呼び出し回避)

レスポンス速度向上

品質は維持 (合意閾値で保証)

1.6 LLM プロバイダー階層

1.6.1 Tier アーキテクチャ

LLM Provider Tiers

Tier 0: ドキュメント参照 (非LLM)

- Context7 - ライブラリドキュメント検索
- Stash - コードベース参照

用途：API仕様、ライブラリ使用方法の参照

↓（必要時参照）

Tier 1: 高速・コスト効率 (Primary Analysis)

- Gemini 2.0 Flash Thinking - 深い推論（実験版）
- Gemini 1.5 Pro - 高品質分析
- Gemini 2.0 Flash - 高速分析

特徴：低コスト、高速、マルチリンガル

用途：初期分析、情報整理、要約

↓

Tier 2: 専門特化 (Specialized Tasks)

Coding専用：

- GPT-5 Codex - OpenAI Codex CLI（適応推論）
- Qwen3 Coder - 480B MoE (35B active, Agentic)

General：

- GPT-5（標準） - 詳細推論・分析

特徴：タスク特化、高品質、ベストプラクティス内蔵

用途：コーディング、デバッグ、専門分析

↓

Tier 3: 標準アグリゲーター (Response Integration)

- Claude Sonnet 4.5 - 標準統合・高速推論

特徴：バランス型、複数回答統合、矛盾解消

用途：通常の複雑度タスクの最終統合

↓

Tier 4: 複雑タスク専用 (Complex Aggregation)

- Claude Opus 4.1 - 最高品質統合

特徴： 最高品質、複雑な論理統合、アーキテクチャ設計

用途： 高複雑度タスク、セキュリティ監査、設計判断

1.6.2 Tier 選択マトリクス

タスクタイプ × 複雑度マトリクス

	低複雑度 (score < 1)	中複雑度 (score 1-2)	高複雑度 (score 2)
Coding	Tier 2: • GPT-5 Codex • Qwen3 Coder Aggregator: Sonnet 4.5	Tier 2: • GPT-5 Codex • Qwen3 Coder Aggregator: Sonnet 4.5	Tier 2+3: • GPT-5 Codex • Qwen3 Coder • Gemini Pro Aggregator: Opus 4.1
Analysis	Tier 1: • Gemini Flash • GPT-5 Aggregator: Sonnet 4.5	Tier 1+2: • GPT-5 • Gemini Pro Aggregator: Sonnet 4.5	Tier 1+2+3: • Gemini Deep • GPT-5 • Sonnet 4.5 Aggregator: Opus 4.1
Basic	Tier 1: • Gemini Flash Aggregator: Sonnet 4.5	Tier 1+2: • Gemini Pro • GPT-5 Aggregator: Sonnet 4.5	Tier 1+2: • Gemini Deep • GPT-5 Aggregator: Opus 4.1
Critical (Security/ Emergency)	Tier 1+2+3: • Gemini Deep • GPT-5 Codex	Tier 1+2+3: • Gemini Deep • GPT-5 Codex	All Tiers: • Gemini Deep • GPT-5 Codex

• Sonnet 4.5	• Sonnet 4.5	• Sonnet 4.5
		• Opus 4.1
Aggregator:	Aggregator:	Aggregator:
Opus 4.1	Opus 4.1	Opus 4.1

1.7 具体的な処理例

1.7.1 例 1: コーディングタスク

入力:

ユーザークエリ: "TypeScriptでEmailバリデーション関数を実装して"
domain: "coding"

処理フロー:

Step 1: タスクタイプ判定

キーワード: "TypeScript", "実装" → Coding
ドメイン: "coding" → Coding確定
→ taskType = "coding"

Step 2: プロバイダー選択

Coding タスク検出
→ Tier 2 Coding モデル選択

Selected:

1. GPT-5 Codex (OpenAI)
2. Qwen3 Coder (OpenRouter)
3. Gemini 2.0 Flash (検証用)

Step 3: 並列実行

GPT-5 Codex

Qwen3 Coder

Gemini Pro

Response 1:

```
```ts
export function
isValidEmail(
 email: string
): boolean {
 ...
}
```
```

confidence: 0.88

Response 2:

```
```ts
function
validateEmail(
 email: string
): boolean {
 ...
}
```
```

confidence: 0.85

Response 3:

```
```ts
const validate
Email = (
 email: string
) => {
 ...
}
```
```

confidence: 0.80

Step 4: Quorum Judge

類似度計算:

- Response1 vs Response2: 0.82
- Response1 vs Response3: 0.75
- Response2 vs Response3: 0.78

合意グループ:

- Group A: [Response1, Response2] (size=2, similarity=0.82)

Quorum達成: YES (2 k=2)

Consensus Score: 0.82

Step 5: アグリゲーター

複雑度スコア: 0 (低)

→ Aggregator: Claude Sonnet 4.5

統合処理:

- Response1 (GPT-5 Codex)
- Response2 (Qwen3 Coder)

↓

最終回答:

TypeScriptでのEmailバリデーション実装:

```
```typescript
export function isValidEmail(
 email: string
): boolean {
 if (!email) return false;
 const normalized = email.trim();
 if (normalized.length > 254)
 return false;
 return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(normalized);
}
```
```

【実装のポイント】

1. 型安全性: TypeScriptの型定義活用
2. エッジケース: 空文字・長さ制限
3. RFC5322準拠の正規表現パターン

【テスト例】

[テストコード...]

メタデータ:

- confidence: 0.88
- consensus: 0.82
- providersUsed: ["gpt-5-codex", "qwen3-coder"]
- aggregator: "sonnet-4.5"

1.7.2 例 2: 複雑な分析タスク

入力:

ユーザークエリ: "マイクロサービスアーキテクチャへの移行における
セキュリティリスクを詳細に分析し、対策を提案して"
domain: "analysis"

処理フロー:

Step 1: タスクタイプ判定

キーワード: "分析", "セキュリティ" → Analysis
ドメイン: "analysis" → Analysis確定
→ taskType = "analysis"

Step 2: 複雑度評価

score = 0

キーワード含む:

- "architect" → +1
- "security" → +1

日本語キーワード含む:

- "詳細" → +1

プロンプト長: 72文字 (< 500)

疑問符: 0個

総スコア: 3 (高複雑度)

→ complexity = "high"

→ Aggregator: Opus 4.1 (Tier 4)

Step 3: プロバイダー選択

Analysis + High Complexity

→ Tier 1 + Tier 2 + 異なるベンダー

Selected:

1. Gemini 2.0 Flash Thinking (実験)
2. GPT-5 (標準)
3. Claude Sonnet 4.5 (分析用)

Step 4: 並列実行

3プロバイダーが並列で分析実行

各プロバイダーが以下を提供:

- セキュリティリスクの分析
- 対策の提案
- 実装の推奨事項

Step 5: Quorum Judge

3つの回答の類似度評価

- 共通リスク項目の抽出
- 対策の整合性確認
- Consensus Score: 0.78

Step 6: アグリゲーター (Opus 4.1)

高複雑度タスク → Opus 4.1 使用

統合処理:

- 各回答からリスクを網羅的に統合
- 矛盾する推奨事項を調整
- 優先順位付き対策リストを生成

↓

最終回答:

マイクロサービス移行のセキュリティ分析

【主要リスク】

1. サービス間通信の脆弱性
 - API Gateway認証の欠如
 - 暗号化されていない内部通信
2. 分散認証・認可の課題
 - トークン管理の複雑化

- セッション同期の問題

[...]

【優先対策】

Priority 1: (即時実施)

- mTLS導入によるサービス間通信保護
- OAuth2.0 + JWTによる統一認証基盤

Priority 2: (1ヶ月以内)

- APIゲートウェイでのレート制限
- サービスメッシュ導入検討

[...]

【実装ロードマップ】

[詳細なステップ...]

メタデータ:

- confidence: 0.92
- consensus: 0.78
- providersUsed: ["gemini-2.0-flash-thinking-exp", "gpt-5", "sonnet-4.5"]
- aggregator: "opus-4.1"

1.8 品質保証の仕組み

1.8.1 品質メトリクス

品質スコアの算出方法

1. Confidence (信頼度) - 0.0 ~ 1.0

各プロバイダーが自己評価で算出:

- 回答の確実性
- 情報源の信頼性

- 推論の論理的－貫性

算出例：

```
confidence = (
    reasoning_quality * 0.4 +
    source_reliability * 0.3 +
    logical_coherence * 0.3
)
```

2. Consensus（合意度）－ 0.0 ～ 1.0

プロバイダー間の回答類似度：

```
consensus = max(
    各合意グループのサイズ
) / 総プロバイダー数
```

例：3プロバイダー中2つが合意

→ consensus = 2/3 = 0.67

3. Coherence（－貫性）－ 0.0 ～ 1.0

回答内の論理的－貫性：

- 主張と根拠の整合性
- 矛盾の有無
- 結論の妥当性

4. Completeness（完全性）－ 0.0 ～ 1.0

回答の網羅性：

- 質問への完全な回答
- 必要な情報の包含
- フォローアップの提供

```
最終品質スコア = (  
    confidence * 0.35 +  
    consensus * 0.35 +  
    coherence * 0.15 +  
    completeness * 0.15  
)
```

1.8.2 品質閾値と処理フロー

品質判定フローチャート

ステップ1: Consensus チェック

IF consensus < 0.6:

- 追加プロバイダー実行
- 最大 maxProviders まで繰り返し

ELSE IF consensus ≥ 0.6:

- ステップ2へ

ステップ2: Confidence チェック

IF avg_confidence < 0.7:

- 警告付きで応答
- メタデータに低信頼度フラグ

ELSE IF avg_confidence ≥ 0.7:

- ステップ3へ

ステップ3: アグリゲーション

- 合意回答を統合
- 最終品質スコア算出
- メタデータ付与

ステップ4: 最終チェック

IF final_quality_score < 0.65:

- 品質警告を含めて応答
- ユーザーに再試行を提案

ELSE:

- 高品質応答として返却

1.8.3 品質保証のベストプラクティス

品質保証のための設計原則

1. 最低プロバイダー数の保証

minProviders 2 (常に)

Critical タスクは 3-4 プロバイダー

2. ベンダー多様性

異なるベンダーのLLMを混在

OpenAI → Google → Anthropic

ベンダー固有バイアスを相殺

3. Tier ベースの役割分担

Tier 1/2: 専門分析

Tier 3/4: 統合・品質向上

4. Early Stopping による効率化

Quorum達成時に即座に終了

コスト削減とレスポンス速度向上

5. メタデータの完全性

使用プロバイダーの記録

スコア詳細の保存

トレーサビリティの確保

1.9 まとめ

1.9.1 システムの強み

1. 高品質保証: 複数 LLM のコンセンサスにより、単一 LLM より高品質な回答を生成
2. 自動最適化: タスクタイプに応じた最適な LLM 選択で、品質とコストを両立
3. スケーラビリティ: Tier 構造により、新しい LLM の追加が容易
4. トレーサビリティ: すべての判断プロセスをメタデータとして記録

1.9.2 期待される効果

- 回答品質: 単一 LLM 比で 15-20% 向上
 - 一貫性: コンセンサスメカニズムにより安定した品質
 - コスト効率: Early Stopping で 30-40% のコスト削減
 - レスポンス速度: 並列実行で高速化 単一 LLM 比 60-70% の時間
-

1.10 補足資料

1.10.1 関連ドキュメント

- [WALL_BOUNCE_SYSTEM.md](#) - Wall-Bounce 技術詳細
- [LLM_PROVIDERS_GUIDE.md](#) - LLM プロバイダー統合ガイド
- [llm-providers.json](#) - プロバイダー設定
- [GPT5_VS_GPT5_CODEX_ 実証結果.md](#) - GPT-5/Codex 使い分け実証

1.10.2 用語集

- **Wall-Bounce:** 複数 LLM の協調による高品質回答生成手法
 - **Quorum:** 合意形成の最小必要数 k 値
 - **Consensus:** プロバイダー間の合意度スコア
 - **Aggregator:** 複数回答を統合する LLM Tier 3/4
 - **Tier:** LLM の役割・品質に基づく階層分類
 - **Early Stopping:** Quorum 達成時の即時終了による効率化
-

作成者: TechSapo Documentation Team 最終更新: 2025-10-07 バージョン: 1.0